

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-aa3433c23264fe1d2.jsonl`
- SHA-256: `0634927cd55ffe2d35da3a0f8e6f97f106d6b637d428ac79fb8e5863b1daabad`
- Source modified: `2026-06-14T09:37:27+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T09:33:38.202Z)

Implement a focused, mergeable engineering PR in the badminton-highlight-indexer repo (cwd C:/Users/avidu/Projects/badminton-highlight-indexer): a PER-USER NO-REGRESSION GUARD for the personalization feature ? guarantees a per-user retrain or a baseline upgrade never makes a user WORSE than their current incumbent.

WHY: the strategy analysis flagged that a deliberately-tuned per-user model "has no shared baseline to regress against." This builds that missing per-user regression baseline, mirroring the existing shared-golden discipline.

READ FIRST (use the real APIs): backend/eval/regression.py (the existing no-regression pattern + its baseline JSON + how it gates), backend/eval/experiment.py (Variant/compare/evaluate_variant/predict), backend/eval/eval_stats.py (paired_delta_ci, paired_sign_test), and the just-merged backend/eval/per_user_eval.py + backend/eval/per_user_gate.py (compose with these, don't duplicate).

CONSTRUCT ? add backend/eval/per_user_regression.py: given a user's videos (experiment.VideoCandidates), a FROZEN incumbent CONTROL (the user's currently-active variant/config ? which for a brand-new user IS the shipped baseline) and a CANDIDATE (the proposed new per-user variant after retrain/upgrade), evaluate both on the user's held-out videos and return ACCEPT only if the candidate does NOT regress: accept iff candidate is not significantly worse than control (reuse eval_stats ? a regression = the paired ?F1 CI excludes 0 on the WRONG side / candidate clearly worse). On a tie / insufficient evidence ? KEEP THE INCUMBENT (conservative; never accept a risky swap). Key a small per-user baseline record by a `user\_id::stage` string (user_id nullable; NULL = local single-user ? design-for-north-star / implement-for-current). Provide save/load of that per-user baseline record as JSON (sibling to regression_baseline.json's approach). Return {accept: bool, keep_incumbent: bool, reason, control_f1, candidate_f1, mean_delta, ci_low, ci_high, regressed: bool}. Pure-CPU, deterministic, no GPU/torch.

TESTS (tests/test_per_user_regression.py, GPU-free synthetic VideoCandidates a la test_fusion_compare.py): ACCEPTS a clear improvement; REJECTS a clear regression (keep incumbent); tie / insufficient evidence ? keep incumbent; the user_id::stage baseline record round-trips through save/load (tmp_path).

VERIFY: `python -m ruff check backend/eval/per\_user\_regression.py tests/test\_per\_user\_regression.py`; `python -m mypy backend/eval/per\_user\_regression.py`; `python -m pytest tests/test\_per\_user\_regression.py tests/test\_per\_user\_eval.py tests/test\_per\_user\_gate.py tests/test\_eval\_stats.py -q`. All pass.

Branch-safe ISOLATED worktree off origin/master: `git -C C:/Users/avidu/Projects/badminton-highlight-indexer fetch origin` then `git -C ... worktree add C:/Users/avidu/Projects/bhi-pureg -b feat/per-user-no-regression origin/master`. Commit explicit paths (end msg with `Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>`); push; `gh pr create --base master`; DO NOT merge. Return: PR number/URL, files, exact ruff/mypy/pytest results, worktree path.

2. assistant (2026-06-14T09:33:41.070Z)

I'll implement this PR. Let me start by reading the existing APIs to understand the patterns I need to compose with.

3. user (2026-06-14T09:33:44.011Z)

(Bash completed with no output)

4. user (2026-06-14T09:33:44.023Z)

```
1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7      ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8      predictions  ? the production pipeline's stored DB rows      [backend.eval.harness]
9      comparison   ? interval matching + P/R/F1                    [backend.eval.metrics]
10     baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
39         different labels."""
40         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
41         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
```

```

54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66         # ----- baseline store
67
68         def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69             """Load the per-video baseline snapshot file (returns {} if absent)."""
70             if not os.path.isfile(path):
71                 return {}
72             with open(path) as f:
73                 return json.load(f)
74
75
76         def _baseline_key(video_id: str, stage: str) -> str:
77             return f"{video_id}::{stage}"
78
79
80         def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                           f1: float, path: str = DEFAULT_BASELINE_PATH,
82                           iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                           gt_fingerprint: Optional[str] = None) -> None:
84             """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86             Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87             later run computed under different settings (or against different labels) is
detected
88             as incommensurable rather than silently compared.
89             """
90             baselines = load_baselines(path)
91             baselines[_baseline_key(video_id, stage)] = {
92                 "video_id": video_id, "stage": stage,
93                 "precision": precision, "recall": recall, "f1": f1,
94                 "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95             }
96             os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97             with open(path, "w") as f:
98                 json.dump(baselines, f, indent=2, sort_keys=True)
99             logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)
100
101
102         # ----- the runner
103
104         def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                    stage: str = "final", iou_threshold: float = 0.5,
106                    detector: Optional[str] = None,
107                    tolerance: float = DEFAULT_TOLERANCE,
108                    baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109             """Score stored predictions for one video against ground truth and compare to
baseline.
110
111             `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112             AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113             precision OR recall falls more than `tolerance` below the snapshotted baseline.
114             If no baseline exists yet, `regressed` is False (nothing to compare to) and the

```

```

115 caller can choose to snapshot the current numbers via `save_baseline`.
116 """
117 preds = get_predictions(db, video_id, stage, detector=detector)
118 s = metrics.score(preds, ground_truth, iou_threshold)
119 fp = gt_hash(ground_truth)
120
121 baselines = load_baselines(baseline_path)
122 b = baselines.get(_baseline_key(video_id, stage))
123
124 reasons: List[str] = []
125 regressed = False
126 has_baseline = b is not None
127 base_p = base_r = None
128 if b is not None:
129     base_p, base_r = b["precision"], b["recall"]
130     # Incommensurable comparison guard: a baseline taken at a different IoU/detector
131     or
132     # against different labels is not a valid reference ? flag it instead of
133     trusting it.
134     b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
135     b.get("gt_hash")
136     if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
137         regressed = True
138         reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
139         (incommensurable)")
140     if b_det is not None and b_det != detector:
141         regressed = True
142         reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
143         (incommensurable)")
144     if b_hash is not None and b_hash != fp:
145         regressed = True
146         reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
147         changed)")
148     if s.precision < base_p - tolerance:
149         regressed = True
150         reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
151         {tolerance}")
152     if s.recall < base_r - tolerance:
153         regressed = True
154         reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
155         {tolerance}")
156     else:
157         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
158
159 return RegressionResult(
160     video_id=video_id, stage=stage, iou_threshold=iou_threshold,
161     precision=s.precision, recall=s.recall, f1=s.f1,
162     baseline_precision=base_p, baseline_recall=base_r,
163     tolerance=tolerance, regressed=regressed, reasons=reasons,
164     has_baseline=has_baseline, gt_hash=fp,
165 )
166
167 def regress_with_provider(db: Database, video_id: str, video_ref: str,
168     tier: str = "file", stage: str = "final",
169     iou_threshold: float = 0.5,
170     detector: Optional[str] = None,
171     tolerance: float = DEFAULT_TOLERANCE,
172     baseline_path: str = DEFAULT_BASELINE_PATH,
173     guideline: Optional[str] = None,
174     sport: str = "badminton",
175     provider_kwargs: Optional[dict] = None) -> RegressionResult:
176     """End-to-end: obtain ground truth from the `tier` annotation provider, then
177     regress.
178 
```

```

171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172     once those providers land). `video_ref` is whatever that provider expects (a CSV
173     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174     predictions in the DB (the file MD5).
175     """
176     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                 detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
188             tol={r.tolerance}"
189             return head + ".join(f"\n    - {why}" for why in r.reasons)
190

```

5. user (2026-06-14T09:33:44.039Z)

```

1     """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3     The owner adopted personalization as a flag-gated feature on a generalization-
first
4     baseline: a cue/variant B may be flipped ON for one user only when that user's own
5     held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6     fall back to the shipped baseline (variant A). This module is the decision logic for
that
7     flip; it is pure (no I/O) and reuses backend.eval.eval_stats for every statistic
8     (paired_delta_ci + paired_sign_test) so the math is never reinvented.
9
10    The counter-case this guards against (and its mitigations, implemented here):
11
12    1. Minimum-N floor. Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13        (see eval_stats C1). Refuse to promote below min_n videos ? point estimates
from
14        too few held-out videos are noise, not signal.
15    2. Honest lift, not a single mean. Promote only when the paired ?F1 95% bootstrap CI
16        excludes 0 and the paired sign test agrees (majority of videos win, p <=
threshold).
17        This is the global promotion bar applied per user.
18    3. Structural guards are never disabled on "no lift". A structural guard (e.g.
Gate-A
19        net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20        modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21        that simply lacks those failure cases must NEVER turn the guard OFF. For
structural=True
22        the decision is therefore always "keep ON", independent of the measured delta.
23
24    Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25    default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26    threaded through). Nothing here changes existing eval behaviour; callers opt in.
27    """
28    from __future__ import annotations
29
30    from dataclasses import dataclass, field

```

```

31     from typing import Any, Dict, Sequence
32
33     from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35     __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
43         structural
44         cue that cleared the full bar).
45         - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
46         this is
47         always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
48         equals
49         ``promote``.
50         - ``reason`` ? short human-readable explanation of the decision.
51         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
52         tests)
53         so a caller can log/audit exactly why a user was or wasn't personalized.
54         """
55
56         promote: bool
57         keep_on: bool
58         reason: str
59         n: int
60         mean_delta: float
61         ci_low: float
62         ci_high: float
63         ci_excludes_zero: bool
64         sign_test: Dict[str, float] = field(default_factory=dict)
65         floor_met: bool = False
66         structural_protected: bool = False
67
68         def as_dict(self) -> Dict[str, Any]:
69             """Plain-dict view (for JSON logging / telemetry)."""
70             return {
71                 "promote": self.promote,
72                 "keep_on": self.keep_on,
73                 "reason": self.reason,
74                 "n": self.n,
75                 "mean_delta": self.mean_delta,
76                 "ci_low": self.ci_low,
77                 "ci_high": self.ci_high,
78                 "ci_excludes_zero": self.ci_excludes_zero,
79                 "sign_test": self.sign_test,
80                 "floor_met": self.floor_met,
81                 "structural_protected": self.structural_protected,
82             }
83
84     def should_promote_for_user(
85         a_scores: Sequence[float],
86         b_scores: Sequence[float],
87         *,
88         structural: bool = False,
89         min_n: int = 5,
90         p_threshold: float = 0.05,
91         confidence: float = 0.95,
92         n_boot: int = 2000,
93         seed: int = 0,
94         eps: float = 0.0,

```

```

92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
scores.
94         ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
for
95         variant A (control / shipped baseline) and variant B (with the cue), aligned by
video
96         (same order). The decision rules (faithful to the module docstring):
97
98         1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
100         ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
value is
101         in failure modes the user's corpus may not yet contain.
102         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
paired
103         ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
videos
104         win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
105
106         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
``paired_sign_test``);
107         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108         """
109         n = min(len(a_scores), len(b_scores))
110         floor_met = n >= min_n
111
112         # --- Evidence (reuse eval_stats; never reinvent the statistics)
-----
113         if n == 0:
114             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
115             sign_test: Dict[str, float] = paired_sign_test([], eps)
116             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
117         else:
118             delta = paired_delta_ci(
119                 a_scores, b_scores,
120                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
121             )
122             mean_delta = float(delta["mean_delta"])
123             ci_low = float(delta["ci_low"])
124             ci_high = float(delta["ci_high"])
125             ci_excludes_zero = bool(delta["ci_excludes_zero"])
126             sign_test = delta["sign_test"]
127
128             sign_p = float(sign_test.get("p_value", 1.0))
129             sign_wins = float(sign_test.get("wins", 0.0))
130             sign_losses = float(sign_test.get("losses", 0.0))
131             # The sign test "agrees" with a promotion only if it is significant AND the majority
132             # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133             # never satisfy the promotion bar.
134             sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136             # --- Rule 1: structural guards are never disabled on "no lift"
-----
137             if structural:
138                 return PromotionDecision(
139                     promote=False,          # nothing to "promote" ? it ships ON by design
140                     keep_on=True,          # ALWAYS kept ON, regardless of measured per-user
lift
141                     reason=(
142                         "structural guard kept ON regardless of measured per-user lift "
143                         "(value is in failure modes this user's corpus may not yet contain)"
144                     ),
145                     n=n,

```

```

146         mean_delta=mean_delta,
147         ci_low=ci_low,
148         ci_high=ci_high,
149         ci_excludes_zero=ci_excludes_zero,
150         sign_test=sign_test,
151         floor_met=floor_met,
152         structural_protected=True,
153     )
154
155     # --- Rule 2: minimum-N floor (refuse before even looking at lift)
156     -----
157     if not floor_met:
158         return PromotionDecision(
159             promote=False,
160             keep_on=False,          # fall back to the shipped baseline (variant A)
161             reason=(
162                 f"below minimum-N floor (n={n} < min_n={min_n}); "
163                 "falling back to shipped baseline"
164             ),
165             n=n,
166             mean_delta=mean_delta,
167             ci_low=ci_low,
168             ci_high=ci_high,
169             ci_excludes_zero=ci_excludes_zero,
170             sign_test=sign_test,
171             floor_met=False,
172             structural_protected=False,
173         )
174
175     # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
176     -----
177     if ci_excludes_zero and sign_agrees:
178         promote = True
179         reason = (
180             f"promoted: n={n} >= {min_n}, ?F1 95% CI "
181             f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
182             f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
183             {p_threshold})"
184         )
185     else:
186         promote = False
187         misses = []
188         if not ci_excludes_zero:
189             misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
190         if not sign_agrees:
191             misses.append(
192                 f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
193                 f"p={sign_p:.4f}"
194             )
195         reason = (
196             "insufficient evidence (" + "; ".join(misses) + "); "
197             "falling back to shipped baseline"
198         )
199
200     return PromotionDecision(
201         promote=promote,
202         keep_on=promote,          # non-structural: keep_on tracks promote exactly
203         reason=reason,
204         n=n,
205         mean_delta=mean_delta,
206         ci_low=ci_low,
207         ci_high=ci_high,
208         ci_excludes_zero=ci_excludes_zero,
209         sign_test=sign_test,
210         floor_met=True,

```



```

208         structural_protected=False,
209     )
210

```

6. user (2026-06-14T09:33:44.040Z)

```

1     """Held-out-user LEARNING-CURVE eval ? "<K videos to superior quality" as a release
gate.
2
3     The personalization feature's promise is operational, not aspirational: *after the user
4     contributes K of their own videos, the personalized (candidate) recipe should beat the
5     generic (control) one on the user's STILL-UNSEEN footage.* This module turns that
promise
6     into a falsifiable criterion by sweeping the corpus size ``k`` and asking, at each
``k``,
7     **does the candidate beat control on a held-out split, honestly?**
8
9     It is the per-user analogue of `fusion_compare`: pure-CPU, no new scoring/stats math.
Every
10    fold reuses the shipped experiment primitives ?
11
12    - `experiment.predict` ? variant ? held-out intervals (static variants need no fit);
13    - `experiment._train` ? fit a learned variant on the corpus split (the same honest
14      train-on-others step `evaluate_variant`'s LOVO uses, here train-on-the-k-corpus);
15    - `metrics.score` ? per-video held-out F1;
16    - `eval_stats.paired_delta_ci` ? the honest paired ?F1 (bootstrap CI + exact sign
test, C1).
17
18    **Split semantics (held-out USER split, NOT leave-one-out).** At each ``k`` the first
``k``
19    videos are the *personalization corpus* (fit/operating-point source) and the remaining
20    ``len(videos) - k`` are *held-out* (scored, never seen during fit). This mirrors the
product
21    reality ? the user has uploaded ``k`` videos and we want quality on their next one ? and
keeps
22    ``k`` and the evaluation set cleanly disjoint, so a measured lift can't be in-sample.
23
24    The headline ``smallest_k_with_lift`` is the smallest ``k`` at which the candidate beats
25    control with the **CI clearing 0 AND the sign test agreeing** (B wins on more held-out
videos
26    than it loses) ? the same promotion-grade bar `compare()`'s honest block uses, never a
bare
27    mean. ``None`` means the criterion was never met on this corpus (the honest negative).
28
29    Pure, offline, deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
30    `VideoCandidates`). See `docs/EXPERIMENT_HARNESS.md` +
`docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
31    """
32    from __future__ import annotations
33
34    from typing import Any, Dict, List, Optional, Sequence
35
36    from backend.eval import eval_stats as _stats
37    from backend.eval import experiment
38    from backend.eval.classifier import LogisticRegression
39    from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
40    from backend.eval.metrics import score
41
42
43    def _fit_model(variant: Variant, corpus: Sequence[VideoCandidates],
44                  iou: float) -> Optional[LogisticRegression]:
45        """The fitted model a variant needs to predict held-out videos, or None if it needs
none.
46
47        - **static variant** (no learned filter): None ? `predict` scores directly, no

```

```

leakage.
48 - **pinned model** (``classifier_model`` set): the shipped model, loaded as-is (no
per-fold
49 training ? reports "how the shipped artifact does on this user's held-out
footage").
50 - **learned recipe** (``feature_names``, no pinned model): fit on the ``corpus``
split via
51 the harness's own honest training step. The corpus and held-out sets are disjoint,
so the
52 held-out scores are genuinely out-of-sample.
53 """
54 if variant.classifier_model is not None:
55     return LogisticRegression.load(variant.classifier_model)
56 if variant.feature_names:
57     return experiment._train(variant, corpus, iou)
58 return None
59
60
61 def _heldout_f1s(variant: Variant, corpus: Sequence[VideoCandidates],
62                 heldout: Sequence[VideoCandidates], iou: float) -> List[float]:
63     """Per-held-out-video F1 for ``variant``, fit on ``corpus`` (if learned), scored on
64     ``heldout``. Held-out videos are video-aligned with the returned list (caller pairs
65     B?A)."""
66     model = _fit_model(variant, corpus, iou)
67     f1s: List[float] = []
68     for vc in heldout:
69         assert vc.gts is not None # guarded in learning_curve before any scoring
70         preds = experiment.predict(variant, vc, model=model)
71         f1s.append(score(preds, vc.gts, iou).f1)
72     return f1s
73
74 def learning_curve(videos: Sequence[VideoCandidates],
75                   control_variant: Variant,
76                   candidate_variant: Variant,
77                   *,
78                   iou: float = 0.5,
79                   min_k: int = 2,
80                   seed: int = 0) -> Dict[str, Any]:
81     """Sweep corpus size ``k`` and report where the candidate first honestly beats
control.
82
83 For one user's ``videos``, for each ``k`` in ``min_k .. len(videos) - 1``: fit/score
both
84 variants on the first ``k`` videos (the personalization corpus) and evaluate on the
remaining
85 held-out videos. Per-held-out-video F1s are paired B?A and summarised with
86 ``eval_stats.paired_delta_ci`` (mean ?F1 + bootstrap CI + exact sign test).
87
88 Returns ``{n_videos, min_k, iou, per_k, smallest_k_with_lift}`` where each ``per_k``
record is::
89
90     {k, n_heldout, control_f1, candidate_f1,
91      mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}
92
93 ``control_f1`` / ``candidate_f1`` are the mean held-out F1 of each variant at that
``k``.
94 ``smallest_k_with_lift`` is the smallest ``k`` whose record shows a real lift ?
candidate
95 above control, the ?F1 CI clearing 0, AND the sign test agreeing (B wins on strictly
more
96 held-out videos than it loses) ? else ``None`` (the honest "never wins" answer).
97
98 Pure + deterministic given ``seed`` (the only randomness is the CI bootstrap).
Raises

```

```

99         `ExperimentError` on unlabeled videos or a too-small corpus (no valid ``k`` to
sweep).
100         """
101         n = len(videos)
102         for vc in videos:
103             if vc.gts is None:
104                 raise ExperimentError(
105                     f"{vc.name} has no golden labels; learning_curve needs labeled held-out
videos")
106         if min_k < 1:
107             raise ExperimentError(f"min_k must be >= 1 (got {min_k})")
108         if n <= min_k:
109             # Every k in [min_k, n-1] needs k >= min_k corpus videos AND >= 1 held-out
video, so
110             # n must exceed min_k. Too-few videos ? no sweepable k; report an empty, honest
curve.
111             return {
112                 "n_videos": n,
113                 "min_k": min_k,
114                 "iou": iou,
115                 "per_k": [],
116                 "smallest_k_with_lift": None,
117             }
118
119         per_k: List[Dict[str, Any]] = []
120         smallest_k_with_lift: Optional[int] = None
121         for k in range(min_k, n):
122             corpus = videos[:k]
123             heldout = videos[k:]
124             control_f1s = _heldout_f1s(control_variant, corpus, heldout, iou)
125             candidate_f1s = _heldout_f1s(candidate_variant, corpus, heldout, iou)
126
127             paired = _stats.paired_delta_ci(control_f1s, candidate_f1s, seed=seed)
128             sign_test = paired["sign_test"]
129             # Honest lift = the candidate is above control AND the CI clears 0 AND the sign
test
130             # agrees (B wins on strictly more held-out videos than it loses) ? the promotion
bar.
131             control_mean = sum(control_f1s) / len(control_f1s) if control_f1s else 0.0
132             candidate_mean = sum(candidate_f1s) / len(candidate_f1s) if candidate_f1s else
0.0
133             has_lift = (
134                 candidate_mean > control_mean
135                 and bool(paired["ci_excludes_zero"])
136                 and paired["mean_delta"] > 0.0
137                 and sign_test["wins"] > sign_test["losses"]
138             )
139             per_k.append({
140                 "k": k,
141                 "n_heldout": len(heldout),
142                 "control_f1": control_mean,
143                 "candidate_f1": candidate_mean,
144                 "mean_delta": paired["mean_delta"],
145                 "ci_low": paired["ci_low"],
146                 "ci_high": paired["ci_high"],
147                 "ci_excludes_zero": bool(paired["ci_excludes_zero"]),
148                 "sign_test": sign_test,
149             })
150             if has_lift and smallest_k_with_lift is None:
151                 smallest_k_with_lift = k
152
153         return {
154             "n_videos": n,
155             "min_k": min_k,
156             "iou": iou,

```

```

157         "per_k": per_k,
158         "smallest_k_with_lift": smallest_k_with_lift,
159     }
160

```

7. assistant (2026-06-14T09:33:46.945Z)

Now let me read the remaining key APIs: `experiment.py`, `eval\_stats.py`, and the existing tests for patterns.

8. user (2026-06-14T09:33:47.361Z)

```

1     """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3     The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
4     a
5     single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
6     training
7     and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
8     ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
9     turns those into an honest summary:
10
11     - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
12       interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
13       many *videos* it wins on (the distribution-free test the repo already leans on:
14       "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
15       noise.
16     - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
17       learned
18       producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
19       match, never a held-out window from the same video).
20
21     See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
22     only),
23     deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
24     ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
25     """
26     from __future__ import annotations
27
28     import math
29     import random
30     import statistics
31     from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
32
33     Interval = Tuple[float, float]
34     FitPredict = Callable[[List[int], List[int]], List[Any]]
35
36     def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
37                      n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
38         """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
39
40         ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
41         to
42         resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
43         band.
44         """
45         vals = [float(v) for v in values]
46         n = len(vals)
47         if n == 0:
48             return (0.0, 0.0)
49         if n == 1:
50             return (vals[0], vals[0])
51         rng = random.Random(seed)
52         means: List[float] = []

```

```

47     for _ in range(max(1, n_boot)):
48         means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49     means.sort()
50     alpha = (1.0 - confidence) / 2.0
51     last = len(means) - 1
52     lo = means[max(0, int(math.floor(alpha * last)))]
53     hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54     return (lo, hi)
55
56
57 def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                 n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59     """`{mean, std, n, ci_low, ci_high, confidence}` ? report this, never a bare mean
60     (C1)."""
61     vals = [float(v) for v in values]
62     n = len(vals)
63     mean = statistics.mean(vals) if vals else 0.0
64     std = statistics.pstdev(vals) if n > 1 else 0.0
65     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
66     return {"mean": mean, "std": std, "n": float(n),
67            "ci_low": lo, "ci_high": hi, "confidence": confidence}
68
69 def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70     """Two-sided exact sign test over per-fold deltas (B ? A).
71
72     ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73     excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74     distribution-free "did B win on enough *videos*?" check.
75     """
76     wins = sum(1 for d in deltas if d > eps)
77     losses = sum(1 for d in deltas if d < -eps)
78     ties = len(deltas) - wins - losses
79     n_eff = wins + losses
80     if n_eff == 0:
81         p = 1.0
82     else:
83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87            "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96     by
97     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98     sign
99     test agrees ? not when a single mean ticked up.
100     """
101     n = min(len(a_values), len(b_values))
102     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103     summary = mean_with_ci(deltas, confidence, n_boot, seed)
104     return {
105         "mean_delta": summary["mean"],
106         "ci_low": summary["ci_low"],
107         "ci_high": summary["ci_high"],
108         "n": float(n),
109         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110         "sign_test": paired_sign_test(deltas, eps),

```

```

109     }
110
111
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
114         unit.
115         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116         first-seen (deterministic). Groups with no complement (a single group) yield no
117         fold.
118         """
119         seen: List[Any] = []
120         for g in groups:
121             if g not in seen:
122                 seen.append(g)
123         n = len(groups)
124         folds: List[Tuple[List[int], List[int]]] = []
125         for g in seen:
126             test = [i for i in range(n) if groups[i] == g]
127             train = [i for i in range(n) if groups[i] != g]
128             if test and train:
129                 folds.append((train, test))
130         return folds
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
133     List[Optional[Any]]:
134         """Out-of-fold predictions for honest stacking (C3).
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
136         the
137         other groups and returns predictions for ``test_idx`` (in that order); results are
138         scattered back to a list aligned to the input items. So a *learned* producer's
139         outputs,
140         when used as arbiter features, never include in-sample predictions ? the leakage
141         path
142         that inflates measured lift on a tiny corpus. Items in a singleton group stay
143         ``None``
144         (needs >= 2 groups to predict).
145         """
146         n = len(groups)
147         oof: List[Optional[Any]] = [None] * n
148         for train_idx, test_idx in grouped_folds(groups):
149             preds = fit_predict(train_idx, test_idx)
150             for pos, ti in enumerate(test_idx):
151                 if pos < len(preds):
152                     oof[ti] = preds[pos]
153         return oof

```

9. user (2026-06-14T09:33:47.974Z)

```

1     """GPU-free tests for the held-out-user learning-curve eval
2     (backend.eval.per_user_eval).
3
4     Mirrors the synthetic-VideoCandidates pattern in test_fusion_compare.py: a learned
5     candidate
6     variant weights a single feature column, a static control variant keeps everything. The
7     held-out-split sweep then answers "after how many of a user's videos does the
8     personalized
9     recipe honestly beat the generic one?" ? found only when the ?F1 CI clears 0 AND the
10    sign test
11    agrees. Pure-CPU (numpy only), deterministic.
12    """
13    import pytest

```

```

10
11 from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
12 from backend.eval.per_user_eval import learning_curve
13
14 SEP = "players_in_court" # the one separating feature the learned candidate weights
15
16 # Per-k record keys the honest curve must carry (incl. the C1 CI + sign-test fields).
17 _EXPECTED_KEYS = {
18     "k", "n_heldout", "control_f1", "candidate_f1",
19     "mean_delta", "ci_low", "ci_high", "ci_excludes_zero", "sign_test",
20 }
21 _SIGN_TEST_KEYS = {"wins", "losses", "ties", "n_effective", "p_value"}
22
23
24 def _video(name: str, *, separating: bool = True) -> VideoCandidates:
25     """A 4-candidate video: 2 positives (match the gts) + 2 negatives (junk).
26
27     ``separating`` True ? the feature is high on rallies / low on junk (true signal);
28     ``separating`` False ? the feature is INVERTED (high on junk), which mistrains a
model
29     fit on too few videos. Held-out videos are always true-signal so they can be scored.
30     """
31     intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
32     sep = [4.0, 4.0, 0.0, 0.0] if separating else [0.0, 0.0, 4.0, 4.0]
33     gts = [(0.0, 10.0), (20.0, 30.0)]
34     return VideoCandidates(name=name, intervals=intervals, features={SEP: sep}, gts=gts)
35
36
37 def _control() -> Variant:
38     """Static control: no learned filter, no gate ? keeps every candidate (over-
fires)."""
39     return Variant(name="generic")
40
41
42 def _candidate() -> Variant:
43     """Learned candidate: weights the separating feature, fit per corpus split (LOVO-
style)."""
44     return Variant(name="personalized", feature_names=[SEP])
45
46
47 def test_candidate_wins_above_some_k():
48     """A corpus where the candidate clearly wins above some k ? smallest_k_with_lift
found.
49
50     The first corpus video carries an INVERTED signal, so a model fit at k=2 misfilters
(no
51     lift); once the true-signal videos outvote it (k>=3) the candidate cleanly beats
control.
52     """
53     videos = [_video("v0", separating=False)] + [_video(f"v{i}") for i in range(1, 7)]
54     res = learning_curve(videos, _control(), _candidate(), iou=0.5, min_k=2, seed=0)
55
56     k = res["smallest_k_with_lift"]
57     assert k is not None
58     assert res["min_k"] <= k < res["n_videos"] # a sensible, in-range corpus
size
59     assert k == 3 # deterministic: lift emerges
at k=3
60
61     # At the winning k the candidate is strictly above control with the honest bar met.
62     rec = next(r for r in res["per_k"] if r["k"] == k)
63     assert rec["candidate_f1"] > rec["control_f1"]
64     assert rec["mean_delta"] > 0.0
65     assert rec["ci_excludes_zero"] is True
66     assert rec["sign_test"]["wins"] > rec["sign_test"]["losses"]

```

```

67
68
69 def test_candidate_never_wins_returns_none():
70     """A corpus where the feature carries NO signal (constant) ? smallest_k_with_lift is
None."""
71     videos = [
72         VideoCandidates(name=f"v{i}",
73                         intervals=[(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0,
51.0)],
74                         features={SEP: [1.0, 1.0, 1.0, 1.0]},          # constant ? non-
separating
75                         gts=[(0.0, 10.0), (20.0, 30.0)])
76     for i in range(6)
77     ]
78     res = learning_curve(videos, _control(), _candidate(), iou=0.5, min_k=2, seed=0)
79
80     assert res["smallest_k_with_lift"] is None
81     # Every k records a non-winning fold: no delta and a CI that does not clear 0.
82     for rec in res["per_k"]:
83         assert rec["mean_delta"] == pytest.approx(0.0)
84         assert rec["ci_excludes_zero"] is False
85
86
87 def test_per_k_records_carry_honest_fields():
88     """Each per-k record carries the expected keys + a well-formed CI and sign test."""
89     videos = [_video(f"v{i}") for i in range(6)]
90     res = learning_curve(videos, _control(), _candidate(), iou=0.5, min_k=2, seed=0)
91
92     assert res["per_k"]          # min_k=2, n=6 ? ks 2..5
93     assert [r["k"] for r in res["per_k"]] == [2, 3, 4, 5]
94     for rec in res["per_k"]:
95         assert set(rec) == _EXPECTED_KEYS
96         assert set(rec["sign_test"]) == _SIGN_TEST_KEYS
97         assert rec["n_heldout"] == res["n_videos"] - rec["k"]
98         # CI brackets the mean delta (±fp slack), as eval_stats.paired_delta_ci
guarantees.
99         assert rec["ci_low"] - 1e-9 <= rec["mean_delta"] <= rec["ci_high"] + 1e-9
100         assert isinstance(rec["ci_excludes_zero"], bool)
101
102
103 def test_too_few_videos_handled_gracefully():
104     """n <= min_k ? no sweepable k: an empty curve + None headline, never a crash."""
105     one = [_video("solo")]
106     two = [_video("a"), _video("b")]
107     for videos in (one, two):
108         res = learning_curve(videos, _control(), _candidate(), min_k=2, seed=0)
109         assert res["per_k"] == []
110         assert res["smallest_k_with_lift"] is None
111         assert res["n_videos"] == len(videos)
112     # zero videos is likewise graceful
113     assert learning_curve([], _control(), _candidate(), min_k=2)["per_k"] == []
114
115
116 def test_unlabeled_video_raises():
117     """A video with no golden labels can't be a held-out target ? ExperimentError."""
118     videos = [_video("v0"), VideoCandidates(name="unlabeled", intervals=[(0.0, 1.0)],
119                                             features={SEP: [1.0]}, gts=None)]
120     with pytest.raises(ExperimentError):
121         learning_curve(videos, _control(), _candidate(), min_k=2)
122

```

10. assistant (2026-06-14T09:33:51.493Z)

Now let me read the `experiment.py` module to understand `Variant`, `VideoCandidates`, `predict`, `evaluate\_variant`, and `\_train`. Let me also look at the test for fusion_compare to

mirror the synthetic pattern.

11. user (2026-06-14T09:33:52.642Z)

```
1      """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3      The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4      non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5      report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6      """
7      import json
8      import os
9      from types import SimpleNamespace
10
11     import pytest
12
13     from backend.eval import fusion_compare
14     from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cand = []
21         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
22             shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES}          # constant
-> non-separating
23             person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24             person["players_in_court"] = 4.0 if pos else 0.0              # the
separating signal
25             person["in_court_ratio"] = 1.0 if pos else 0.0
26             cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27             data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                     "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29             with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30                 json.dump(data, f)
31
32
33     def test_fusion_compare_reports_person_driven_lift(tmp_path):
34         d = str(tmp_path)
35         for k in range(6):
36             _write_video(d, f"vid{k}")
37         videos = fusion_compare.load_videos(d)
38         assert len(videos) == 6
39         res = fusion_compare.compare(videos, iou=0.5)
40         assert res["num_videos"] == 6
41         assert res["variant_b"]["honest_lovo"] is True                    # LOVO-honest learned
variant
42         # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43         assert res["delta"]["f1"] > 0.0
44         assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45         # format_result must not crash and must mention both variants.
46         out = fusion_compare.format_result(res, 0.5)
47         assert "shuttle-only" in out and "+person" in out
48
49
50     def test_fusion_compare_delta_stats_and_ci(tmp_path):
51         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""
```

```

52     d = str(tmp_path)
53     for k in range(6):
54         _write_video(d, f"vid{k}")
55     res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
56     ds = fusion_compare.delta_stats(res)
57     assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
58     assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9    # CI
brackets mean ( $\pm$ fp)
59     assert ds["sign_test"]["wins"] == 6          # fusion wins on every video (person
feature separates)
60     assert ds["ci_excludes_zero"] is True
61     out = fusion_compare.format_result(res, 0.5)
62     assert "CI" in out and "sign test" in out
63
64
65     def test_fusion_compare_needs_two_videos(tmp_path):
66         _write_video(str(tmp_path), "only")
67         videos = fusion_compare.load_videos(str(tmp_path))
68         assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69
70
71     # ----- extractor assembly
72
73     def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74         from backend.eval import fusion_golden
75         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
76
77         intervals = [(0.0, 10.0), (40.0, 41.0)]
78         x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79         points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))
80                     for i in range(10)]
81
82         monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                             lambda traj, fps, fw: (intervals, x_shuttle))
84         monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
85         monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
86
87         class _StubDetector:
88             def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
89                 return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
90                             "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
91                         for (sf, ef) in windows]
92
93         spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
94                "labels": "foo.rallies.csv"}
95         data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)
96
97         assert data["name"] == "t"
98         assert data["gts"] == [(0.0, 10.0)]
99         assert len(data["candidates"]) == 2
100        c0 = data["candidates"][0]
101        assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
102        assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
103        assert c0["label"] == 1          # (0,10) matches the gt -> positive
104        assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
105        assert all(isinstance(v, float) for v in c0["person"].values())
106
107
108     def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
109         """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""

```

```

110     from backend.eval import fusion_golden
111     out_dir = str(tmp_path / "out")
112     os.makedirs(out_dir)
113     manifest = [
114         {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
115         {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
116     ]
117     manifest_path = str(tmp_path / "m.json")
118     with open(manifest_path, "w", encoding="utf-8") as f:
119         json.dump(manifest, f)
120     # a prior run already produced alpha's JSON
121     with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
encoding="utf-8") as f:
122         json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)
123
124     extracted = []
125
126     def _stub(spec, **kw):
127         extracted.append(spec["name"])
128         return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
129
130     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
131
132     fusion_golden.run(manifest_path, out_dir, fresh=False)
133     assert extracted == ["beta"] # alpha skipped (already extracted)
134
135     extracted.clear()
136     fusion_golden.run(manifest_path, out_dir, fresh=True)
137     assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140     def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141         """--smallest-first orders by trajectory size; default keeps manifest order."""
142         from backend.eval import fusion_golden
143         out_dir = str(tmp_path / "out")
144         os.makedirs(out_dir)
145         (tmp_path / "big.csv").write_text("x" * 2000)
146         (tmp_path / "small.csv").write_text("x" * 20)
147         manifest = [
148             {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
149             {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
150         ]
151         mp = str(tmp_path / "m.json")
152         with open(mp, "w", encoding="utf-8") as f:
153             json.dump(manifest, f)
154
155         order = []
156
157         def _stub(spec, **kw):
158             order.append(spec["name"])
159             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
160
161         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
162
163         fusion_golden.run(mp, out_dir, smallest_first=True)
164         assert order == ["small", "big"] # smaller trajectory first
165
166         order.clear()

```

```

167         fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
168         assert order == ["big", "small"]          # manifest order preserved
169
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
173         assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
and Avi.MP4"
174         assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
"a.mp4"
175         with pytest.raises(ValueError):
176             _derive_video_path({"name": "n", "labels": "weird.csv"})
177

```

12. user (2026-06-14T09:33:52.827Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. No new scoring math lives
7     here; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32      (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34    Pure + offline + unit-tested. The only DB-touching helper is
35    `load_video_candidates`, which reads persisted candidates via
36    `Database.query_candidate_segments`.
37    """
38    from __future__ import annotations
39
40    import json
41    import logging
42    import os
43    from dataclasses import dataclass, field
44    from datetime import datetime, timezone
45    from hashlib import sha1
46    from typing import Any, Callable, Dict, List, Optional, Sequence
47
48    from backend.eval.calibration import pct_improvement
49    from backend.eval.classifier import LogisticRegression

```

```

50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
ratio
56 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
scores
57
58 logger = logging.getLogger(__name__)
59
60 # Where a comparison run appends one reproducible record. Tracked location (NOT under
61 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100     - ``feature_names`` ? the persisted features the learned filter consumes. When set
101       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102       (honest) ? the recipe, not a pinned artifact.
103     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0

```

```

112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with ``load_video_candidates``, or directly in tests.

```

```

172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213
214 def predict(variant: Variant, vc: VideoCandidates,
215            model: Optional[LogisticRegression] = None,
216            threshold: Optional[float] = None,
217            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
218     """Variant prediction: gate by persisted features, optionally filter by a learned
219     model, then merge. Pure and deterministic.
220
221     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
222     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
223     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
224     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
225     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
226     them.
227
228     """
229     mask = _gate_mask(variant, vc)
230     if variant.feature_names:
231         if model is None:
232             if variant.classifier_model is None:
233                 raise ExperimentError(
234                     f"variant {variant.name!r} is learned but has no model to predict "
235                     f"with (pass a fitted model or set classifier_model)")
236                 model = LogisticRegression.load(variant.classifier_model)

```

```

234         proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
235         if calibrator is not None:
236             proba = [calibrator(p) for p in proba]
237             thr = variant.threshold if threshold is None else threshold
238             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
239             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240             return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                             train_videos: Sequence[VideoCandidates], iou: float
245                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248         (use the variant default). Honest: never looks at the held-out video.
249         """
250         calibrator: Optional[Callable[[float], float]] = None
251         if variant.calibrate:
252             raw: List[float] = []
253             y: List[int] = []
254             for vc in train_videos:
255                 raw.extend(float(p) for p in
256                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
257                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
258             if variant.calibrate == "platt":
259                 calibrator = fit_platt(raw, y)
260             elif variant.calibrate == "isotonic":
261                 calibrator = fit_isotonic(raw, y)
262             else:
263                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264             threshold: Optional[float] = None
265             if variant.tune_threshold:
266                 best_t, best_f1 = variant.threshold, -1.0
267                 for k in range(1, 20):                                     # grid 0.05..0.95 on the train
fold's rally F1
268                     t = round(0.05 * k, 2)
269                     f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                                 vc.gts or [], iou).f1 for vc in train_videos]
271                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                     if mean_f1 > best_f1:
273                         best_f1, best_t = mean_f1, t
274                     threshold = best_t
275             return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))

```



```

291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
303         training, so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (``classifier_model`` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
306         to report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
308         ? train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
310         only).
311         """
312         for vc in videos:
313             if vc.gts is None:
314                 raise ExperimentError(f"{vc.name} has no golden labels; use
315                 compare_unlabeled")
316
317         per_video: List[Dict[str, Any]] = []
318         predictions: Dict[str, List[Interval]] = {}
319
320         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
321         pinned_model = (LogisticRegression.load(variant.classifier_model)
322                         if variant.classifier_model is not None else None)
323         all_model = None
324         if recipe_lovo and not honest:
325             all_model = _train(variant, videos, iou)
326
327         for i, vc in enumerate(videos):
328             cal: Optional[Callable[[float], float]] = None
329             thr: Optional[float] = None
330             if recipe_lovo and honest:
331                 others = [v for j, v in enumerate(videos) if j != i]
332                 if not others:
333                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
334                     pinned model")
335                 model: Optional[LogisticRegression] = _train(variant, others, iou)
336                 if variant.tune_threshold or variant.calibrate: # operating point from
337                     the train fold
338                     assert model is not None # _train never returns
339                     None
340                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
341             elif recipe_lovo: # honest=False ? one model over all
342                 videos
343                 model = all_model
344             else: # static variant or pinned model
345                 model = pinned_model
346                 preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
347                 assert vc.gts is not None # guarded at function top
348                 sc = score(preds, vc.gts, iou)
349                 per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
350                 for m in _METRICS}})
351                 predictions[vc.name] = preds
352

```

```

346     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                 for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
399         }
400
401

```

```

402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,

```

```

455         agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471
472     """
473     preds_a = predict(variant_a, video)
474     preds_b = predict(variant_b, video)
475     mr = match_intervals(preds_a, preds_b, agree_iou)
476
477     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478               for m in mr.matches]
479     agree_ious = [m.iou for m in mr.matches]
480     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
482
483     def _dur(ivs: List[Interval]) -> float:
484         return sum(e - s for s, e in ivs)
485
486     return {
487         "video": video.name,
488         "agree_iou": agree_iou,
489         "variant_a": variant_a.name,
490         "variant_b": variant_b.name,
491         "num_a": len(preds_a),
492         "num_b": len(preds_b),
493         "num_agreed": len(agreed),
494         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
495         "duration_a": _dur(preds_a),
496         "duration_b": _dur(preds_b),
497         "agreed": agreed,
498         "a_only": a_only,
499         "b_only": b_only,
500     }
501
502     # ----- leaderboard / DB
503
504     def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                          timestamp: Optional[str] = None) -> Dict[str, Any]:
506         """Append a compact, reproducible record of a `compare()` result to the JSON
507         leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508         `regression_baseline.json`; deliberately the size of a baseline file, not a service
509         (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510
511         """
512         row: Dict[str, Any] = {
513             "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
514             "iou": result.get("iou"),
515             "label_set_hash": result.get("label_set_hash"),
516             "num_videos": result.get("num_videos"),
517             "variant_a": {"name": result["variant_a"]["variant"],
518                           "spec_hash": result["variant_a"]["spec_hash"],
519                           "aggregate": result["variant_a"]["aggregate"]},

```

```

519         "variant_b": {"name": result["variant_b"]["variant"],
520                        "spec_hash": result["variant_b"]["spec_hash"],
521                        "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
525     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
526     honest = result.get("honest") or {}
527     if honest.get("paired_delta_f1"):
528         row["honest_delta_f1"] = honest["paired_delta_f1"]
529     entries: List[Dict[str, Any]] = []
530     if os.path.isfile(path):
531         with open(path, encoding="utf-8") as f:
532             entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%0.3f) -> %s",
538                row["variant_a"]["name"], row["variant_b"]["name"],
539                (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``duration`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

13. assistant (2026-06-14T09:33:56.268Z)

I now have a complete understanding of the APIs. Let me check the `eval\_baselines` directory and existing baseline JSON format, then set up the isolated worktree.

14. user (2026-06-14T09:33:57.822Z)

```

experiment_leaderboard.json
gemini_wrapper_2026-06-10.json
heuristic_lovo_n6.json
---
```

1b5af67 feat(eval): held-out-user learning-curve eval ? "<K videos to superior quality" as a release gate (#142)

15. assistant (2026-06-14T09:34:00.744Z)

Now let me create the isolated worktree off `origin/master`.

16. user (2026-06-14T09:34:02.133Z)

Preparing worktree (new branch 'feat/per-user-no-regression')
branch 'feat/per-user-no-regression' set up to track 'origin/master'.
HEAD is now at 1b5af67 feat(eval): held-out-user learning-curve eval ? "<K videos to superior quality" as a release gate (#142)

17. assistant (2026-06-14T09:34:05.492Z)

Let me verify the worktree has the just-merged per_user files and the test_per_user_gate.py test (to mirror its patterns).

18. user (2026-06-14T09:34:07.625Z)

```
C:/Users/avidu/Projects/bhi-pureg/backend/eval/per_user_eval.py
C:/Users/avidu/Projects/bhi-pureg/backend/eval/per_user_gate.py
---tests---
C:/Users/avidu/Projects/bhi-pureg/tests/test_eval_stats.py
C:/Users/avidu/Projects/bhi-pureg/tests/test_per_user_eval.py
C:/Users/avidu/Projects/bhi-pureg/tests/test_per_user_gate.py
```

19. user (2026-06-14T09:34:07.784Z)

```
1      """Tests for backend.eval.per_user_gate ? the per-user personalization promotion gate.
2
3      GPU-free, fully synthetic per-video scores, deterministic (the bootstrap seed is
4      threaded
5      through ``should_promote_for_user``). Pins the four safety properties of the counter-
6      case
7      mitigations:
8
9      - promotes a non-structural cue on a clear, above-floor win;
10     - refuses below the minimum-N floor *even with* a positive point estimate (small-N
11     lies);
12     - refuses when the ?F1 CI spans 0 / the sign test disagrees (the realistic n=6 near-
13     null);
14     - **never** disables a structural guard, even given a measured per-user LOSS;
15     - falls back to the shipped baseline (``keep_on=False``) whenever it refuses to promote.
16     """
17     from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
18
19     def test_promotes_clear_win_above_floor():
20         """n=8, B beats A on every video by a healthy margin ? promote ON."""
21         a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54, 0.56, 0.53]
22         b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66, 0.69, 0.65]
23         d = should_promote_for_user(a, b, min_n=5, seed=0)
24         assert isinstance(d, PromotionDecision)
25         assert d.promote is True
26         assert d.keep_on is True          # non-structural: keep_on tracks promote
27         assert d.floor_met is True
28         assert d.ci_excludes_zero is True
29         assert d.mean_delta > 0.0
30         assert d.sign_test["wins"] == 8.0 and d.sign_test["losses"] == 0.0
31         assert d.structural_protected is False
```

```

31 def test_refuses_below_floor_even_with_positive_point_estimate():
32     """n=3 with a strongly positive point estimate is still REFUSED ? small-N lies.
33
34     The floor is checked before lift, so even though this CI would 'exclude 0' the gate
must
35     fall back to the baseline rather than personalize off too few videos.
36     """
37     a = [0.50, 0.55, 0.60]
38     b = [0.65, 0.70, 0.72]          # B wins all 3, big positive mean delta
39     d = should_promote_for_user(a, b, min_n=5, seed=0)
40     assert d.mean_delta > 0.0       # point estimate genuinely looks good ...
41     assert d.promote is False       # ... but we refuse on too-few videos
42     assert d.keep_on is False       # ? fall back to shipped baseline
43     assert d.floor_met is False
44     assert d.n == 3
45     assert "floor" in d.reason.lower()
46
47
48 def test_refuses_n6_near_null_ci_spans_zero_and_sign_test_disagrees():
49     """The realistic n=6 regime: tiny mixed deltas ? CI spans 0 and sign test is 3-3."""
50     a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54]
51     b = [0.51, 0.53, 0.62, 0.50, 0.59, 0.53] # deltas ~ +.01, -.02, +.02, -.02, +.01, -.01
52     d = should_promote_for_user(a, b, min_n=5, seed=0)
53     assert d.floor_met is True      # above the floor ? refusal is on the evidence,
not n
54     assert d.ci_excludes_zero is False
55     assert d.sign_test["wins"] == 3.0 and d.sign_test["losses"] == 3.0
56     assert d.promote is False
57     assert d.keep_on is False      # fall back to baseline
58     assert d.structural_protected is False
59
60
61 def test_refuses_when_only_sign_test_fails():
62     """Guard the AND: even a CI that excludes 0 must not promote if the sign test
disagrees.
63
64     A measured net *loss* (B worse on every video) has a CI that excludes 0 but leans
the
65     wrong way ? the gate must not read 'significant' as 'promote'.
66     """
67     a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]
68     b = [0.50, 0.52, 0.55, 0.51, 0.53, 0.54, 0.56, 0.52] # B loses every video
69     d = should_promote_for_user(a, b, min_n=5, seed=0)
70     assert d.ci_excludes_zero is True      # CI clears 0 ...
71     assert d.mean_delta < 0.0              # ... but on the losing side
72     assert d.sign_test["wins"] == 0.0 and d.sign_test["losses"] == 8.0
73     assert d.promote is False
74     assert d.keep_on is False
75
76
77 def test_structural_guard_never_disabled_even_on_measured_loss():
78     """structural=True ? always keep_on, regardless of a measured per-user LOSS.
79
80     Gate-A net-crossings is the held-shuttle FP killer; its value is in failure modes a
given
81     user's corpus may not contain. A measured loss on such a corpus must NOT turn it
OFF.
82     """
83     a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]
84     b = [0.50, 0.52, 0.55, 0.51, 0.53, 0.54, 0.56, 0.52] # B clearly worse here
85     d = should_promote_for_user(a, b, structural=True, min_n=5, seed=0)
86     assert d.keep_on is True             # NEVER disabled on 'no lift' / loss
87     assert d.promote is False            # nothing to 'promote' ? ships ON by
design
88     assert d.structural_protected is True

```

```

89         assert d.mean_delta < 0.0                # evidence is surfaced, decision
ignores it
90         assert "structural" in d.reason.lower()
91
92
93     def test_structural_guard_kept_on_below_floor_too():
94         """A structural guard stays ON even when the user has too few videos to measure
lift."""
95         a = [0.60, 0.62]
96         b = [0.40, 0.41]                # tiny corpus, B looks worse
97         d = should_promote_for_user(a, b, structural=True, min_n=5, seed=0)
98         assert d.keep_on is True
99         assert d.promote is False
100        assert d.structural_protected is True
101
102
103    def test_empty_input_falls_back_to_baseline():
104        """No paired videos ? no evidence ? fall back to the shipped baseline (non-
structural)."""
105        d = should_promote_for_user([], [], min_n=5, seed=0)
106        assert d.n == 0
107        assert d.floor_met is False
108        assert d.promote is False
109        assert d.keep_on is False
110
111
112    def test_decision_as_dict_round_trips_fields():
113        """as_dict() exposes the full structured result for telemetry/audit logging."""
114        a = [0.50, 0.55, 0.60, 0.52, 0.58, 0.54, 0.56, 0.53]
115        b = [0.62, 0.67, 0.71, 0.63, 0.70, 0.66, 0.69, 0.65]
116        d = should_promote_for_user(a, b, min_n=5, seed=0)
117        out = d.as_dict()
118        for key in (
119            "promote", "keep_on", "reason", "n", "mean_delta", "ci_low", "ci_high",
120            "ci_excludes_zero", "sign_test", "floor_met", "structural_protected",
121        ):
122            assert key in out
123            assert out["promote"] is True and out["keep_on"] is True
124

```

20. assistant (2026-06-14T09:34:17.925Z)

I now have everything I need. Let me design the module.

Key design decisions:

- **Module purpose**: Given a user's videos, a frozen incumbent CONTROL variant, and a CANDIDATE variant, evaluate both on the user's held-out videos and ACCEPT only if the candidate does NOT regress.
- **Evaluation**: Use `experiment.evaluate_variant` to get per-video held-out F1 for each variant, then `eval_stats.paired_delta_ci` for the paired ?F1 (candidate ? control). A regression = the CI excludes 0 on the wrong side (candidate clearly worse). Accept iff candidate is NOT significantly worse.
- **Conservative tie-break**: tie / insufficient evidence ? keep incumbent. But accepting requires "not significantly worse" ? so a tie should ACCEPT only if it's an improvement-or-equal? Re-reading: "accept iff candidate is not significantly worse than control" and "On a tie / insufficient evidence ? KEEP THE INCUMBENT (conservative; never accept a risky swap)."

This is subtle. Let me reconcile: the guard is a **no-regression** guard. The promotion decision (whether the candidate is **better**) is `per_user_gate`'s job. This guard's job: never make a user worse. So:

- If candidate is **significantly worse** (regression) ? REJECT, keep incumbent.
- If candidate is **significantly better** ? ACCEPT (clear improvement).
- If **tie / insufficient evidence** (CI spans 0) ? keep incumbent (conservative, don't swap on noise).

So `accept` = candidate significantly better (CI excludes 0 on the right side AND sign test agrees)`. `regressed` = candidate significantly worse`. The tie case is neither accept nor regressed ? keep incumbent. This matches the test requirements exactly: accept clear improvement, reject clear regression (keep incumbent), tie ? keep incumbent.

Now let me write the module.

21. assistant (2026-06-14T09:35:11.801Z)

```
"""Per-user NO-REGRESSION guard ? the missing per-user analogue of `regression.py`.
```

```
The shared golden set has a disciplined no-regression baseline (`regression.py` +  
`regression_baseline.json`): a candidate is gated so a global change can never silently  
make the *shared* corpus worse. Personalization has no such rail ? a deliberately-tuned  
per-user model "has no shared baseline to regress against", so a per-user retrain or a  
baseline upgrade could quietly make ONE user worse than the model they're already running.  
This module builds that missing per-user regression baseline, mirroring the shared discipline.
```

```
The decision it makes is deliberately narrow and CONSERVATIVE ? it is a *guarantee against  
regression*, not a promotion decision (that is `per_user_gate.should_promote_for_user`):
```

```
* **CONTROL** = the user's *frozen incumbent* ? the variant/config currently active FOR THAT  
  USER. For a brand-new user with nothing personalized yet, the incumbent simply IS the  
  shipped baseline. This is the thing we must never regress below.  
* **CANDIDATE** = the proposed replacement (a per-user retrain, or a global baseline upgrade  
  rolled out to this user).
```

```
Both are evaluated on the user's own held-out videos (`experiment.evaluate_variant`, LOVO-honest  
for a learned recipe ? no new scoring math), paired by video, and summarised with the shipped  
honest statistics (`eval_stats.paired_delta_ci` ? bootstrap ?F1 CI + exact sign test). Then:
```

```
* **regression** ? the candidate is *significantly worse*: the paired ?F1 (candidate ?  
control)  
  95% CI excludes 0 on the WRONG side (`ci_high < 0`). ? REJECT, keep the incumbent.  
* **clear improvement** ? the candidate is *significantly better*: the CI excludes 0 on the  
  right side AND the sign test agrees (candidate wins on strictly more videos than it loses).  
  ? ACCEPT the swap.  
* **tie / insufficient evidence** ? the CI spans 0 (the small-N near-null regime): there is no  
  evidence the swap helps, so we never take the risk. ? KEEP THE INCUMBENT.
```

```
So we only ACCEPT a swap we can *defend* and only ever toward a measured improvement; everything  
ambiguous keeps the incumbent. That is strictly stronger than "don't regress" (it also refuses  
risk-neutral churn), which is exactly the safety the personalization feature was missing.
```

```
A small per-user baseline record is keyed by a `user_id::stage` string (`user_id` nullable ?  
`None` = the local single-user deployment; design-for-north-star, implement-for-current) and  
saved/loaded as JSON, a sibling to `regression.py`'s `regression_baseline.json` approach.
```

```
Pure-CPU, deterministic given `seed` (the only randomness is the CI bootstrap); no GPU / torch  
/
```

```
cv2 / file I/O beyond the explicit baseline save/load. See `docs/EXPERIMENT_HARNESS.md` +  
`docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import logging
```

```
import os
```

```
from dataclasses import asdict, dataclass, field
```

```
from typing import Any, Dict, List, Optional, Sequence
```

```
from backend.eval import eval_stats as _stats
```

```
from backend.eval import experiment
```

```
from backend.eval.experiment import Variant, VideoCandidates
```

```
logger = logging.getLogger(__name__)
```

```
__all__ = [  
    "DEFAULT_PER_USER_BASELINE_PATH",  
    "PerUserRegressionResult",  
    "PerUserBaseline",  
    "baseline_key",  
    "load_baselines",  
    "save_baseline",  
    "evaluate_no_regression",  
]
```

Tracked location (NOT under the gitignored output/), sibling to regression.DEFAULT_BASELINE_PATH so a fresh checkout / CI can load a committed baseline. NULL user_id (local single-user) keys as the literal "local".

```
DEFAULT_PER_USER_BASELINE_PATH = os.path.join("eval_baselines", "per_user_baseline.json")
```

----- result

```
@dataclass  
class PerUserRegressionResult:  
    """The per-user no-regression decision plus the evidence behind it.  
  
    - ``accept`` ? swap the candidate in for this user (only ever ``True`` on a *defended*  
      improvement: the ?F1 CI clears 0 on the right side AND the sign test agrees).  
    - ``keep_incumbent`` ? the user keeps their frozen incumbent (control). Always the exact  
      complement of ``accept`` ? a swap is taken or it is not.  
    - ``regressed`` ? the candidate is *significantly worse* than the incumbent (CI excludes 0  
      on the wrong side). A True here always implies ``accept=False``.  
    - ``reason`` ? short human-readable explanation, for logging / audit.  
    - the remaining fields surface the paired evidence (per-variant mean held-out F1, mean ?F1,  
      CI bounds, and the sign test) so a caller can audit exactly why the swap was/ wasn't  
      taken.  
    """  
  
    accept: bool  
    keep_incumbent: bool  
    reason: str  
    n: int  
    control_f1: float  
    candidate_f1: float  
    mean_delta: float  
    ci_low: float  
    ci_high: float  
    ci_excludes_zero: bool  
    regressed: bool  
    sign_test: Dict[str, float] = field(default_factory=dict)  
  
    def as_dict(self) -> Dict[str, Any]:  
        """Plain-dict view (for JSON logging / telemetry)."""  
        return asdict(self)
```

----- per-user baseline store

```
@dataclass  
class PerUserBaseline:  
    """A snapshotted per-user incumbent record ? the per-user analogue of a regression baseline.  
  
    Keyed by ``user_id::stage``. ``user_id`` is nullable (``None`` ? the local single-user
```

```

deployment, keyed as the literal ``"local"``). ``control_spec_hash`` pins WHICH incumbent
variant the recorded numbers were measured for, so a later check against a different
incumbent is detectable rather than silently trusted (mirrors regression.py's
iou/detector/gt fingerprint guards).
"""

user_id: Optional[str]
stage: str
control_f1: float
control_spec_hash: str
n: int
iou: float

def as_dict(self) -> Dict[str, Any]:
    return asdict(self)

def baseline_key(user_id: Optional[str], stage: str) -> str:
    """Stable ``user_id::stage`` key. A NULL ``user_id`` (local single-user) keys as
    ``local``."""
    return f"{user_id if user_id is not None else 'local'}::{stage}"

def load_baselines(path: str = DEFAULT_PER_USER_BASELINE_PATH) -> Dict[str, dict]:
    """Load the per-user baseline file (returns ``{}`` if absent)."""
    if not os.path.isfile(path):
        return {}
    with open(path, encoding="utf-8") as f:
        return json.load(f)

def save_baseline(baseline: PerUserBaseline,
                  path: str = DEFAULT_PER_USER_BASELINE_PATH) -> str:
    """Snapshot a user's incumbent numbers as the new per-user baseline, keyed
    ``user_id::stage``.

    Round-trips through ``load_baselines``; returns the key written. Mirrors
    ``regression.save_baseline`` (atomic-ish read-modify-write of the one JSON file).
    """
    baselines = load_baselines(path)
    key = baseline_key(baseline.user_id, baseline.stage)
    baselines[key] = baseline.as_dict()
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(baselines, f, indent=2, sort_keys=True)
    logger.info("Saved per-user baseline %s (control_f1=%.3f n=%d)",
                key, baseline.control_f1, baseline.n)
    return key

```

----- the guard

```

def _mean(xs: Sequence[float]) -> float:
    return sum(xs) / len(xs) if xs else 0.0

def evaluate_no_regression(
    videos: Sequence[VideoCandidates],
    control: Variant,
    candidate: Variant,
    *,
    user_id: Optional[str] = None,
    stage: str = "final",
    iou: float = 0.5,

```

```

honest: bool = True,
p_threshold: float = 0.05,
confidence: float = 0.95,
n_boot: int = 2000,
seed: int = 0,
eps: float = 0.0,
baseline_path: Optional[str] = None,
) -> PerUserRegressionResult:
    """Guard a per-user swap: ACCEPT only if ``candidate`` does NOT regress vs the ``control``.

    ``control`` is the user's FROZEN incumbent (their currently-active variant ? for a
    brand-new user this is just the shipped baseline). ``candidate`` is the proposed
    replacement (a per-user retrain or a baseline upgrade). Both are scored on the user's own
    ``videos`` via ``experiment.evaluate_variant`` (LOVO-honest for a learned recipe ? no new
    scoring math), the per-video held-out F1s are paired by video, and the paired ?F1
    (candidate ? control) is summarised with ``eval_stats.paired_delta_ci`` (bootstrap CI +
    exact sign test). The decision (faithful to the module docstring):

        * **regression** (``ci_high < 0`` ? candidate significantly worse): REJECT, keep
        incumbent.
        * **clear improvement** (CI excludes 0 on the right side AND the sign test agrees ?
        candidate wins on strictly more videos than it loses): ACCEPT the swap.
        * **tie / insufficient evidence** (CI spans 0): KEEP THE INCUMBENT (conservative; never
        accept a risky swap).

    ``user_id``/``stage`` key the baseline record (``user_id=None`` ? local single-user). When
    ``baseline_path`` is given the incumbent's measured numbers are snapshotted there (the swap
    is then evaluated as usual). Deterministic given ``seed``. Raises ``ExperimentError`` (from
    ``evaluate_variant``) on unlabeled videos or a learned recipe with too few videos for LOVO.
    """
    eval_control = experiment.evaluate_variant(videos, control, iou, honest)
    eval_candidate = experiment.evaluate_variant(videos, candidate, iou, honest)

    # Per-video held-out F1, video-aligned (evaluate_variant iterates ``videos`` in order, so
    # both lists are pairable by position ? exactly how honest_summary pairs B ? A).
    control_f1s: List[float] = [p["f1"] for p in eval_control["per_video"]]
    candidate_f1s: List[float] = [p["f1"] for p in eval_candidate["per_video"]]
    n = min(len(control_f1s), len(candidate_f1s))

    control_mean = _mean(control_f1s)
    candidate_mean = _mean(candidate_f1s)

    if n == 0:
        # No paired videos at all ? no evidence either way ? keep the incumbent.
        sign_test: Dict[str, float] = _stats.paired_sign_test([], eps)
        result = PerUserRegressionResult(
            accept=False, keep_incumbent=True,
            reason="no held-out videos to evaluate; keeping incumbent",
            n=0, control_f1=control_mean, candidate_f1=candidate_mean,
            mean_delta=0.0, ci_low=0.0, ci_high=0.0, ci_excludes_zero=False,
            regressed=False, sign_test=sign_test,
        )
    else:
        delta = _stats.paired_delta_ci(
            control_f1s, candidate_f1s,
            confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
        )
        mean_delta = float(delta["mean_delta"])
        ci_low = float(delta["ci_low"])
        ci_high = float(delta["ci_high"])
        ci_excludes_zero = bool(delta["ci_excludes_zero"])
        sign_test = delta["sign_test"]
        sign_p = float(sign_test.get("p_value", 1.0))
        sign_wins = float(sign_test.get("wins", 0.0))
        sign_losses = float(sign_test.get("losses", 0.0))

```

```

# A regression = the candidate is significantly WORSE: the ?F1 CI excludes 0 entirely
# on the wrong side (its whole high end is below 0). Never accept such a swap.
regressed = ci_excludes_zero and ci_high < 0.0
# A defended improvement = the CI clears 0 on the RIGHT side AND the sign test agrees
# (more candidate-wins than losses, significant) ? the same promotion-grade bar the
# global eval uses, applied per user. Everything else is a tie / insufficient evidence.
sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
improved = ci_excludes_zero and ci_low > 0.0 and sign_agrees

if regressed:
    accept = False
    reason = (
        f"REGRESSION: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] "
        f"is entirely below 0 (n={n}); keeping incumbent"
    )
elif improved:
    accept = True
    reason = (
        f"accepted: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] excludes 0 "
        f"(improvement), sign test {int(sign_wins)}-{int(sign_losses)} "
        f"(p={sign_p:.4f} <= {p_threshold}); swapping in candidate"
    )
else:
    accept = False
    reason = (
        f"tie / insufficient evidence: candidate ?F1 95% CI "
        f"[{ci_low:.4f}, {ci_high:.4f}] spans 0 (n={n}); keeping incumbent "
        f"(no defended improvement, never a risky swap)"
    )

result = PerUserRegressionResult(
    accept=accept, keep_incumbent=not accept, reason=reason,
    n=n, control_f1=control_mean, candidate_f1=candidate_mean,
    mean_delta=mean_delta, ci_low=ci_low, ci_high=ci_high,
    ci_excludes_zero=ci_excludes_zero, regressed=regressed, sign_test=sign_test,
)

# Snapshot the incumbent's measured numbers as this user's baseline when asked ? the
# per-user analogue of regression.save_baseline. Does not change the decision above.
if baseline_path is not None:
    save_baseline(
        PerUserBaseline(
            user_id=user_id, stage=stage,
            control_f1=control_mean, control_spec_hash=control.spec_hash(),
            n=n, iou=iou,
        ),
        path=baseline_path,
    )

return result

```

22. user (2026-06-14T09:35:11.816Z)

File created successfully at: C:/Users/avidu/Projects/bhi-pureg/backend/eval/per_user_regression.py (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-14T09:35:17.345Z)

Now let me write the tests. I need synthetic `VideoCandidates` that produce a clear improvement, a clear regression, and a tie. Let me think carefully about the mechanics.

For the static control + learned candidate pattern from `test\_per\_user\_eval.py`: a learned candidate that weights a separating feature beats the over-firing static control ? improvement. To get a regression, I flip it: make the candidate worse than control. The simplest

deterministic approach is two static variants where one over-fires (low precision) and one is tuned.

Let me design concrete videos. With `min\_players` gate (Gate B on `players\_in\_court`), I can build a control that keeps everything (over-fires ? low F1) vs a candidate that gates to high players (good F1) = improvement. And reverse for regression. Let me verify the F1 mechanics by reasoning, then run.

24. assistant (2026-06-14T09:35:45.702Z)

```
"""GPU-free tests for the per-user NO-REGRESSION guard (backend.eval.per_user_regression).
```

```
Mirrors the synthetic-`VideoCandidates` pattern in test_fusion_compare.py /
```

```
test_per_user_eval.py:
```

```
each video carries 2 positive candidates (match the gts) + 2 negative candidates (junk), with a
`players_in_court` feature that separates rallies (4) from junk (0). Two *static* variants
then
```

```
make the decision deterministic and bootstrap-free in expectation:
```

- a **GOOD** variant gates on `min\_players>=1` ? keeps only the rallies ? perfect F1;
- an **OVERFIRE** variant has no gate ? keeps every candidate ? low F1 (false positives).

```
Swapping OVERFIRE?GOOD is a clear improvement (ACCEPT); GOOD?OVERFIRE is a clear regression
(REJECT, keep incumbent); GOOD?GOOD is a tie (keep incumbent). Pure-CPU, deterministic.
```

```
"""
```

```
import pytest
```

```
from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
```

```
from backend.eval.per_user_regression import (
```

```
    PerUserBaseline,
    PerUserRegressionResult,
    baseline_key,
    evaluate_no_regression,
    load_baselines,
    save_baseline,
```

```
)
```

```
PLAYERS = "players_in_court"
```

```
def _video(name: str) -> VideoCandidates:
```

```
    """A 4-candidate video: 2 rallies (players=4, match the gts) + 2 junk windows
```

```
(players=0)."""
```

```
    intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
```

```
    players = [4.0, 4.0, 0.0, 0.0]
```

```
    gts = [(0.0, 10.0), (20.0, 30.0)]
```

```
    return VideoCandidates(name=name, intervals=intervals, features={PLAYERS: players}, gts=gts)
```

```
def _good() -> Variant:
```

```
    """Gates on players>=1 ? keeps only the two rally windows ? perfect held-out F1."""
```

```
    return Variant(name="good", min_players=1)
```

```
def _overfire() -> Variant:
```

```
    """No gate ? keeps every candidate (2 rallies + 2 junk) ? low precision, low F1."""
```

```
    return Variant(name="overfire")
```

```
def _corpus(n: int = 6):
```

```
    return [_video(f"v{i}") for i in range(n)]
```

```
def test_accepts_clear_improvement():
```

```
    """Candidate (gated, perfect F1) clearly beats the over-firing incumbent ? ACCEPT the
```

```

swap."""
    res = evaluate_no_regression(_corpus(6), control=_overfire(), candidate=_good(), seed=0)
    assert isinstance(res, PerUserRegressionResult)
    assert res.accept is True
    assert res.keep_incumbent is False
    assert res.regressed is False
    assert res.candidate_f1 > res.control_f1
    assert res.mean_delta > 0.0
    assert res.ci_excludes_zero is True
    assert res.ci_low > 0.0 # CI clears 0 on the RIGHT side
    assert res.sign_test["wins"] > res.sign_test["losses"]
    assert "accept" in res.reason.lower()

def test_rejects_clear_regression_keeps_incumbent():
    """Candidate (over-firing) is clearly worse than the gated incumbent ? REJECT, keep incumbent."""
    res = evaluate_no_regression(_corpus(6), control=_good(), candidate=_overfire(), seed=0)
    assert res.accept is False
    assert res.keep_incumbent is True # the user keeps their better incumbent
    assert res.regressed is True
    assert res.candidate_f1 < res.control_f1
    assert res.mean_delta < 0.0
    assert res.ci_excludes_zero is True
    assert res.ci_high < 0.0 # whole CI below 0 ? a real regression
    assert "regression" in res.reason.lower()

def test_tie_insufficient_evidence_keeps_incumbent():
    """Identical variants ? F1 is exactly 0, CI spans 0 ? keep the incumbent (never swap on noise)."""
    res = evaluate_no_regression(_corpus(6), control=_good(), candidate=_good(), seed=0)
    assert res.accept is False
    assert res.keep_incumbent is True
    assert res.regressed is False # a tie is NOT a regression ...
    assert res.mean_delta == pytest.approx(0.0) # ... it is no measurable difference
    assert res.ci_excludes_zero is False
    assert res.candidate_f1 == pytest.approx(res.control_f1)
    assert "insufficient evidence" in res.reason.lower()

def test_no_videos_keeps_incumbent():
    """An empty corpus ? no evidence ? keep the incumbent, never a crash."""
    res = evaluate_no_regression([], control=_good(), candidate=_overfire(), seed=0)
    assert res.n == 0
    assert res.accept is False
    assert res.keep_incumbent is True
    assert res.regressed is False

def test_unlabeled_video_raises():
    """A held-out video with no golden labels can't be scored ? ExperimentError (from evaluate_variant)."""
    videos = [_video("v0"),
               VideoCandidates(name="unlabeled", intervals=[(0.0, 1.0)],
                              features={PLAYERS: [1.0]}, gts=None)]
    with pytest.raises(ExperimentError):
        evaluate_no_regression(videos, control=_good(), candidate=_overfire(), seed=0)

def test_result_as_dict_round_trips_fields():
    """as_dict() exposes the full structured result for telemetry / audit logging."""
    res = evaluate_no_regression(_corpus(6), control=_overfire(), candidate=_good(), seed=0)
    out = res.as_dict()
    for key in (

```

```

    "accept", "keep_incumbent", "reason", "n", "control_f1", "candidate_f1",
    "mean_delta", "ci_low", "ci_high", "ci_excludes_zero", "regressed", "sign_test",
):
    assert key in out
    assert out["accept"] is True and out["keep_incumbent"] is False

```

----- baseline record round-trip

```

def test_baseline_key_null_user_is_local():
    """A NULL user_id (local single-user) keys as the literal 'local'; a real id is
    namespaced."""
    assert baseline_key(None, "final") == "local::final"
    assert baseline_key("u123", "final") == "u123::final"
    assert baseline_key("u123", "shadow") == "u123::shadow"

def test_baseline_record_round_trips_through_save_load(tmp_path):
    """A user_id::stage baseline record survives save ? load unchanged (sibling to
    regression_baseline.json)."""
    path = str(tmp_path / "per_user_baseline.json")
    rec = PerUserBaseline(user_id="u123", stage="final", control_f1=0.875,
                          control_spec_hash="abc123def456", n=6, iou=0.5)
    key = save_baseline(rec, path=path)
    assert key == "u123::final"

    loaded = load_baselines(path)
    assert key in loaded
    assert loaded[key] == rec.as_dict()
    assert loaded[key]["control_f1"] == pytest.approx(0.875)
    assert loaded[key]["user_id"] == "u123"

    # A second user/stage coexists under its own key (read-modify-write, not clobber).
    rec2 = PerUserBaseline(user_id=None, stage="shadow", control_f1=0.5,
                          control_spec_hash="000", n=3, iou=0.5)
    save_baseline(rec2, path=path)
    both = load_baselines(path)
    assert set(both) == {"u123::final", "local::shadow"}
    assert both["local::shadow"]["user_id"] is None

def test_evaluate_can_snapshot_incumbent_baseline(tmp_path):
    """Passing baseline_path snapshots the incumbent's measured numbers without changing the
    decision."""
    path = str(tmp_path / "pu.json")
    res = evaluate_no_regression(_corpus(6), control=_good(), candidate=_overfire(),
                               user_id="u9", stage="final", baseline_path=path, seed=0)
    # decision is unaffected by the snapshot (this is still a regression -> keep incumbent)
    assert res.regressed is True and res.keep_incumbent is True
    saved = load_baselines(path)
    key = baseline_key("u9", "final")
    assert key in saved
    # the snapshot records the INCUMBENT (control) numbers, not the candidate's
    assert saved[key]["control_f1"] == pytest.approx(res.control_f1)
    assert saved[key]["n"] == res.n

```

25. user (2026-06-14T09:35:45.715Z)

File created successfully at: C:/Users/avidu/Projects/bhi-pureg/tests/test_per_user_regression.py (file state is current in your context ? no need to Read it back)

26. assistant (2026-06-14T09:35:47.868Z)

Now let me run the verification commands. First check what Python/tooling is available in the worktree.

27. user (2026-06-14T09:35:49.239Z)

All checks passed!

28. user (2026-06-14T09:36:04.165Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file
```

29. assistant (2026-06-14T09:36:06.098Z)

mypy passes (only informational notes). Now the pytest run.

30. user (2026-06-14T09:36:10.683Z)

```
..... [100%]
30 passed in 2.28s
```

31. assistant (2026-06-14T09:36:13.821Z)

All 30 tests pass. Let me get the exact count breakdown per file and confirm the regression/improvement mechanics held as designed (no flakiness in the bootstrap).

32. user (2026-06-14T09:36:20.760Z)

```
..... [100%]
9 passed in 1.63s
=== verbose new test ===
tests/test_per_user_regression.py::test_accepts_clear_improvement PASSED [ 11%]
tests/test_per_user_regression.py::test_rejects_clear_regression_keeps_incumbent PASSED [ 22%]
tests/test_per_user_regression.py::test_tie_insufficient_evidence_keeps_incumbent PASSED [ 33%]
tests/test_per_user_regression.py::test_no_videos_keeps_incumbent PASSED [ 44%]
tests/test_per_user_regression.py::test_unlabeled_video_raises PASSED [ 55%]
tests/test_per_user_regression.py::test_result_as_dict_round_trips_fields PASSED [ 66%]
tests/test_per_user_regression.py::test_baseline_key_null_user_is_local PASSED [ 77%]
tests/test_per_user_regression.py::test_baseline_record_round_trips_through_save_load PASSED [ 88%]
tests/test_per_user_regression.py::test_evaluate_can_snapshot_incumbent_baseline PASSED [100%]
```

33. assistant (2026-06-14T09:36:25.365Z)

All green. Now let me verify branch safety before committing, then stage explicit paths and commit.

34. user (2026-06-14T09:36:26.727Z)

```
=== branch ===
feat/per-user-no-regression
=== HEAD ===
1b5af67 feat(eval): held-out-user learning-curve eval ? "<K videos to superior quality" as a
release gate (#142)
=== commits ahead of origin/master ===
=== status ===
?? backend/eval/per_user_regression.py
?? tests/test_per_user_regression.py
```

35. assistant (2026-06-14T09:36:29.237Z)

Clean: HEAD is `origin/master`, no sibling commits ahead, only my two untracked files. Let me stage explicit paths and commit.

36. user (2026-06-14T09:36:30.177Z)

```
warning: in the working copy of 'backend/eval/per_user_regression.py', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'tests/test_per_user_regression.py', LF will be replaced by CRLF
the next time Git touches it
A backend/eval/per_user_regression.py
A tests/test_per_user_regression.py
```

37. user (2026-06-14T09:36:40.563Z)

```
[feat/per-user-no-regression 28f43e9] feat(eval): per-user no-regression guard ? never make a
user worse on a retrain/upgrade
2 files changed, 456 insertions(+)
create mode 100644 backend/eval/per_user_regression.py
create mode 100644 tests/test_per_user_regression.py
```

38. assistant (2026-06-14T09:36:43.426Z)

Now push and create the PR.

39. user (2026-06-14T09:36:46.057Z)

```
remote:
remote: Create a pull request for 'feat/per-user-no-regression' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/per-user-no-
regression
remote:
branch 'feat/per-user-no-regression' set up to track 'origin/feat/per-user-no-regression'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      feat/per-user-no-regression -> feat/per-user-no-regression
```

40. user (2026-06-14T09:37:04.789Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/143>

41. assistant (2026-06-14T09:37:07.729Z)

PR created. Let me do a final confirmation of the PR and clean up the worktree listing (leaving the worktree in place per the task ? I'll just report its path).

42. user (2026-06-14T09:37:09.979Z)

```
{"baseRefName":"master","headRefName":"feat/per-user-no-
regression","number":143,"state":"OPEN","title":"feat(eval): per-user no-regression guard ?
never make a user worse on a retrain/upgrade","url":"https://github.com/avidullu/badminton-
highlight-indexer/pull/143"}
```

43. assistant (2026-06-14T09:37:27.022Z)

Done. The PR is open and not merged, as instructed.

Summary

Implemented a **per-user no-regression guard** for the personalization feature ? the missing per-user analogue of the shared-golden no-regression rail (`backend/eval/regression.py`).

PR: #143 ? <https://github.com/avidullu/badminton-highlight-indexer/pull/143> (base `master`,

OPEN, not merged)

Worktree: `C:/Users/avidu/Projects/bhi-pureg` (branch `feat/per-user-no-regression`, branched off `origin/master`, isolated)

Files (both new):

- `backend/eval/per\_user\_regression.py` ? the guard
- `tests/test\_per\_user\_regression.py` ? 9 GPU-free tests

What it does: given a user's `VideoCandidates`, a frozen incumbent **CONTROL** (the user's currently-active variant ? for a brand-new user this IS the shipped baseline) and a proposed **CANDIDATE** (retrain/upgrade), it evaluates both on the user's held-out videos via `experiment.evaluate\_variant` (LOVO-honest), pairs the per-video F1s, and summarises with `eval\_stats.paired\_delta\_ci` / `paired\_sign\_test`. Decision:

- **regression** (?F1 CI entirely below 0) ? REJECT, keep incumbent (`regressed=True`);
- **defended improvement** (CI clears 0 on the right side AND sign test agrees) ? ACCEPT;
- **tie / insufficient evidence** (CI spans 0) ? KEEP THE INCUMBENT (conservative, never a risky swap).

Returns `PerUserRegressionResult{accept, keep\_incumbent, reason, n, control\_f1, candidate\_f1, mean\_delta, ci\_low, ci\_high, ci\_excludes\_zero, regressed, sign\_test}`. A `PerUserBaseline` record is keyed by `user\_id::stage` (`user\_id` nullable; `None` ? `local`) with JSON save/load, sibling to `regression\_baseline.json`. Pure-CPU, deterministic, no new scoring/stats math ? it composes the shipped primitives.

Exact verification results (in the worktree):

- `python -m ruff check backend/eval/per\_user\_regression.py tests/test\_per\_user\_regression.py` ? **All checks passed!**
- `python -m mypy backend/eval/per\_user\_regression.py` ? **Success: no issues found in 1 source file** (only informational `annotation-unchecked` notes from imported modules)
- `python -m pytest tests/test\_per\_user\_regression.py tests/test\_per\_user\_eval.py tests/test\_per\_user\_gate.py tests/test\_eval\_stats.py -q` ? **30 passed in 2.28s** (9 of them the new file)

Branch safety: verified HEAD was `origin/master` with zero sibling commits ahead before committing; staged only the two explicit paths; the dev container has no GPU/torch, so this is the pure-logic path only (the eval primitives are CPU/stdlib, so the suite runs fully here).

One caveat worth flagging: the env note says this container lacks GPU/torch/cv2/numpy, but the test suite ran green here because `experiment`/`eval\_stats` are pure-Python and the synthetic `VideoCandidates` need no numpy. If CI exercises a broader import path, that's the owner's machine-side check ? but these four test files all pass locally.